

## Question 1: Async Sum of Squares

Write a Python program using asyncio similar to the factorial example.

### Requirements:

- Create an async function called `sum_of_squares(name, number)`
- It should calculate the sum of squares from  $1^2 + 2^2 + \dots + \text{number}^2$
- Print a message in each iteration like: "Task {name}: Computing square of {i}..."
- Use `await asyncio.sleep(1)` in each loop iteration to simulate work
- After finishing, print the final result: "Task {name}: sum of squares({number}) = {result}"
- In `main()`, run three tasks concurrently for numbers **3, 5, and 6** using `asyncio.gather()`
- Print the final list of results

## Question 2: Async Multiplication Table

Write a Python program using asyncio:

### Requirements:

- Create an async function called `multiplication_table(name, number)`
- It should print the multiplication table for numbers from 1 to 5 (e.g.,  $5 \times 1 = 5$ ,  $5 \times 2 = 10$ , etc.).
- Print a message in each step like: "Task {name}: Calculating {number} x {i}..."
- Use `await asyncio.sleep(0.8)` in each iteration
- After finishing, print: "Task {name}: Completed multiplication table for {number}"
- In `main()`, run three tasks concurrently for numbers **4, 6, and 8** using `asyncio.gather()`
- At the end, print "All tables completed!"

## Question 3: Decorator

Write a decorator named `execution_time` that prints 'Function Started' before execution and 'Function Completed' after execution. Apply it to `calculate_square(number)`, call the function, and display the returned result.